소프트웨어공학: 프로젝트 계획서

기술개발 과 제	(국문 미스터 대박 디너 서비스) (영문 Mr.Daebak Dinner Service)			
과제팀 이름	알파카앞팔털파카		지도교수	이병정 교수님
개 발 기 간	2025년 9월 ~ 2025년 12월 (총 3개월)			
개발소요비용	총 액	37,266천원)		
과 제 팀 구 성 원	이름	이태권	박주영	최문기
	학과	컴퓨터과학부	컴퓨터과학부	컴퓨터과학부
	학번	-	-	-
	연락처	-	-	-

소프트웨어 프로젝트 과제를 성실히 수행하고자 과제 계획서를 제출합니다.

2025년 9월 21일

과제 수행자1: 이태권 (인)
과제 수행자2: 박주영 (인)
과제 수행자3: 최문기 (인)

1. 개발 과제의 개요

가. 개발 과제 요약

미스터 대박 디너 서비스는 AI 도구과 결합한 주문 및 관리 시스템이다. 고객에게 주문 인터페이스를 제공해 주문의 편의를 제공하고, 직원에게는 직원용 인터페이스를 제공해 재고 관리 및 배달 편의를 제공한다. 주문은 GUI 기반으로 이루어지며, AI 음성인식을 통한 주문 또는 서비스 이용의 편의 제공 또한 가능하다. 고객은 메뉴를 자유롭게 수정할 수 있으며 빠르고 직관적인 조작이 가능하도록 유연한 주문 환경을 제공한다. 이외에도 기본적인 등급을 통한 회원관리 등의 관련 기능들이 있다.

나. 개발 과제의 배경 및 효과

◇ 배경

- 최근 외식 트렌드에서는 단순한 배달을 넘어 집에서 특별한 경험을 원하는 수요가 증가하고 있음. 특히 기념일과 특별한 날에 맞춤형 서비스를 제공할 수 있는 고급 배달 서비스는 차별화된 시장 가치를 가짐.
- 최근 AI 기반 음성인식 기술과 온라인 주문 시스템이 빠르게 확산되고 있음. 기존 배달 서비스들은 UI 중심이지만, AI 음성인식 기반 주문까지 제공하는 통합 모델은 드물어 경쟁 우위를 확보할 수 있음.
- 기존 상용 배달 앱들은 대량 주문이나 패스트푸드 판매에 최적화되어 있음. 따라서 '미스터 대박'과 같은 고급 디너의 복합적 주문 경험을 제대로 지원하지 못하는 기술적 한계가 존재함. 제한적인 주문 옵션, 복잡한 UX, 맞춤형 서빙 스타일 적용의 어려움 등이 주요 문제점으로 나타남. 이를 개선하기 위해 규격화된 범용 앱에서 벗어나 다층적 선택과 자유로운 수정이 가능한 유연한 주문 로직을 갖출 필요가 있음. 나아가 AI 음성인식 등 직관적인 인터페이스를 접목한 서비스 전용 맞춤형 플랫폼 구축이 요구됨.

◇ 효과

- 고객 만족도 증대 : GUI, AI 음성인식 기반 쉽고 편리한 주문과정과, 개인화된 맞춤 주문을 통해 고객에게 최상의 사용자 경험을 제공함. 또한 과거 주문 내역을 활용한 간편 재주문 기능은 단골 고객의 선호도를 강화할 것임.
- 운영 효율성 향상: 주문 접수부터 조리, 배달까지의 과정이 시스템을 통해 간편화되고 체계적으로 관리되어, 주문 누락이나 실수와 같은 인적 오류를 최소화하고 직원들의 업무 효율을 극대화함.
- 데이터 기반 마케팅 : 주문 시, 서비스에는 저장되는 상세한 주문 정보를 분석하여 인기 메뉴를 파악할 수 있으며 이를 기반으로 베스트 셀링 조합을 발굴할 수 있음. 또한 단골 고객을 대상으로 한 맞춤형 할인을 제공하며 데이터 기반의 정교한 마케팅 전략을 수립하여 매출 증대에 기여할 수 있음.

다. 개발 과제의 목표와 내용

◇ 목표

- '미스터 대박'의 모든 비즈니스 로직을 완벽하게 지원하며, GUI와 AI 음성인식을 모두 지원하는 지능형 통합 주문 및 운영 관리 플랫폼 개발

◇ 개발 내용

- 고객용 주문 인터페이스
  - 회원 관리 시스템
    - 고객의 성명, 주소, 연락처 등 개인정보를 저장하고 관리하는 회원가입 및 로그인 기능
    - 회원 정보를 기반으로 한 단골 고객 할인 정책 적용 기능
  - 다중 채널 주문 시스템
    - 1단계 (GUI 인터페이스): 사용자가 웹 또는 앱 환경에서 시각적으로 메뉴(디너 종류, 서빙 스타일)를 선택하고, 장바구니에서 옵션을 쉽게 추가, 변경, 삭제할 수 있는 직관적인 인터페이스를 구현합니다.
    - 2단계 (음성인식 인터페이스): "발렌타인 디너 디럭스 스타일로 주문해 줘" 와 같은 자연어 명령을 AI가 인식하여 주문을 처리하는 기능을 구현합니다. "샴페인 한 병 더 추가하고 커피는 빼줘"와 같은 복합적인 수정 요청도 처리할 수 있도록 개발합니다.
  - 개인화 기능
    - 로그인 시 고객의 최근 주문 목록(주문 시간, 메뉴, 가격, 배달 정보 포함)을 표시하여 손쉬운 재주문이 가능하도록 지원합니다.
    -
- 직원용 관리 인터페이스
  - 주문 관리 대시보드
    - 실시간으로 접수되는 모든 주문을 주방 및 배달 담당 직원이 명확하게 확인할 수 있는 시스템
    - 주문 상태(접수, 조리 중, 배달 중, 배달 완료)를 실시간으로 추적 및 업데이트하는 기능
  - 업부 분배 시스템
    - 접수된 주문의 세부 사항(메뉴, 스타일, 커스텀 요청)을 조리 담당 직원에게 정확히 전달하는 기능
    - 배달 주문을 배달 담당 직원에게 효율적으로 할당하는 기능
- 시스템 백엔드
  - 데이터베이스 설계: 고객 정보, 메뉴 정보, 가격 정책, 주문 규칙, 주문 내역 등을 효율적으로 관리할 수 있는 데이터베이스를 구축
  - 비즈니스 로직 구현: '샴페인 축제 디너'의 스타일 제한, 서빙 스타일에 따른 가격 변동 등 '미스터 대박'만의 고유한 비즈니스 규칙을 시스템에 정확히 프로그래밍

라. 개발 과제의 산출물

◇ 분석산출물

- 요구 사항 정의서, 유스케이스

◇ 설계산출물

- 시스템 아키텍처 설계서, 데이터베이스 설계서, UI/UX 디자인, API 설계서

◇ 구현산출물

- 소스 코드, 테스트 코드, 빌드 스크립트, 성능 테스트 결과, 배포 스크립트

마. 용어 정의

- 주문관련
  - Order(주문) : 고객이 요청한 식사 구매 건
  - 데이터베이스 설계: 고객 정보, 메뉴 정보, 가격 정책, 주문 규칙, 주문 내역 등을 효율적으로 관리할 수 있는 데이터베이스를 구축

- 주문관련
  - Order (주문): 고객이 요청한 식사 구매 건
  - Order Status (주문 상태): 주문의 진행 단계 (결제대기, 승인됨, 조리중, 조리완료, 배달중, 배달완료, 취소됨, 거절됨, 환불됨)
  - Simple Order (빠른 재주문): 과거 주문 내역을 기반으로 동일한 구성으로 재주문하는 기능
- 메뉴/상품관련
  - Dinner Type (디너 타입): 식사의 종류 (발렌타인 디너, 프렌치 디너, 잉글리시 디너, 샴페인 디너)
  - Serving Style (서빙 스타일): 식사의 제공 방식 (심플, 그랜드, 디럭스)
  - Component (구성품): 식사를 구성하는 개별 재료/아이템 (빵, 와인, 샐러드, 스테이크 등)
  - Component Type (구성품 종류): 구성품의 카테고리
- 장바구니 관련
  - Cart (장바구니): 고객이 주문 전 담아두는 임시 저장 공간
  - Cart Menu (장바구니 메뉴): 장바구니에 담긴 개별 메뉴 항목
  - Cart Menu Component (장바구니 메뉴 구성품): 장바구니 메뉴의 구성품 정보
- 재고 관련
  - Stock (재고): 각 구성품의 보유 수량
- 사용자 관련
  - Customer (고객): 주문을 하는 사용자
  - Staff (직원): 주문을 관리하고 재고를 관리하는 관리자
- 음성 주문 관련
  - Voice Order (음성 주문): 음성 인터페이스를 통한 주문 방식
  - Conversation Session (대화 세션): 음성 주문 시 고객과의 대화 맥락을 유지하는 단위
  - Speech-to-Text (음성-텍스트 변환): 음성 입력을 텍스트로 변환하는 기능

2. 완료작품의 평가방법

평가 영역	평가 항목	평가 방법	적용기준	개 발 목표치	비중 (%)
소프트웨어 품질 (ISO/IEC 25010)	기능 적합성 (Functionality)	주문/결제/배달 기능 시나리오 테스트	전체 기능 요구사항 대비 테스트 케이스 통과율	기능 커버리지 100%	15
	신뢰성 (Reliability)	코드 스캐너와 정적 분석 도구로 잠재적 버그 탐지	KLOC(1000줄) 당 버그 수	≤ 5	10
	보안성 (Security)	OWASP Top10 점검, 카드정보 암호화 확인	주요 보안 취약점 발견 여부	주요 취약점 없음	10
Usability / UX (SUS + UEQ-S)	시스템 사용성 (SUS)	SUS 설문 (10문항)	SUS 점수 분포	평균 점수 ≥ 70점	10
	사용자 경험 (UEQ-S)	UEQ-S 설문 (8문항, 매력성·효율성 등)	UEQ-S 평가 차원별 점수	평균 ≥ 1.0 (긍정 영역)	10
성능 테스트 (ISO/IEC 14756)	응답시간 (Response Time)	부하 테스트, 95% 퍼센타일 응답시간 측정	95th percentile 응답시간	≤ 2초 (GUI 기준)	10
	처리량 (Throughput)	JMeter 등으로 초당 주문 처리량 측정	초당 처리 가능 요청 수	≥ 50건/초	10
	자원 활용 (Resource Utilization)	일반적인 경우 서비스가 이루어지는 서버에서의 CPU/메모리 사용률 모니터링	CPU/메모리 사용률 평균치	CPU ≤ 70%, 메모리 ≤ 80%	5
	확장성 (Scalability)	사용자 수 2배 증가 시 성능 측정	성능 저하율	응답시간 ≤ 1.5배 증가	5
음성 인식 지표	문자 오류율 (CER)	한국어 메뉴/명령어 음성 테스트	Character Error Rate (CER)	CER ≤ 10% (조용한 환경)	10
	인식 속도 (Latency)	음성 입력 → 결과 출력까지 측정	입력~출력 사이 평균 지연시간	≤ 2초	5

3. 개발일정 및 추진체계

가. 개발 일정

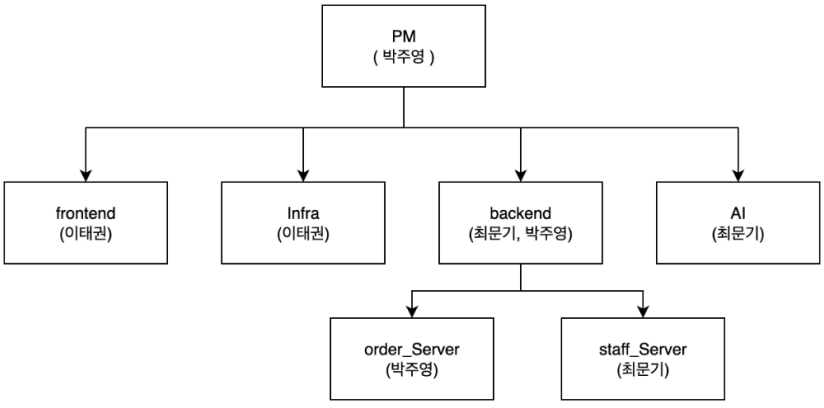
단계별 세부개발 내용	담당자	개발기간 (월단위)				비 고
		9	10	11	12	
요구사항 정의 및 프로젝트 제안서 작성	전원	■■■□				제출 9/22
개발환경 구축	전원	■■■□				
API 명세서 초안 작성	박주영, 최문기	□□□■				
DB 스키마 설계	박주영, 최문기	□□□■				
Figma UI/UX 프로토타입 설계	이태권	□□□■	■□□□			
UML 모델링 및 중간 보고서	전원	□□□■	■■■□			제출 10/20
백엔드 API(직원 인터페이스)	최문기		■■■□			
백엔드 API(주문 인터페이스)	박주영		■■■■	■□□□		
주문 및 직원 인터페이스 GUI 구현	이태권		□■■■	■■□□		
음성인식 모듈 구현 및 연동	최문기		□□□■	■■□□		
통합 및 기능 테스트	전원			□■■□		
최종 버그 수정 및 안정화	전원			□■■■		
최종 발표 및 보고서 작성	전원			□□□■		발표 12/1
유지보수 및 프로젝트 마무리	전원				■■■■	

나. 구성원 및 추진체계

◇ 구성원

- 이태권: 프론트엔드(React) 중심, 필요 시 백엔드 보조, 개발 과정 및 배포 인프라 관리
- 박주영: 프로젝트 매니저, 주문 인터페이스 백엔드(Spring), 프론트 보조
- 최문기: 직원 인터페이스 백엔드(Spring) 및 데이터 처리, AI 음성인식 기능
- 공통: 문서 작성, UML 모델링, 발표 준비

◇ 계통도



4. 위험 분석

위험	발생 가능성	영향도	우선순위	대응방안
팀원 결석/이탈 (연인과의 불화, 건강 등)	Medium	High	Extreme	역할 백업, 문서화 강화, 일정 유연성 확보
팀원 실력 편차로 특정 파트 과부하	Medium	Medium	High	역할 재분배, 교육자료 공유, 페어프로그래밍, 코드리뷰
타 시험·대회 준비로 일정 지연	High	High	Extreme	학사·대회 일정 반영, 핵심 기능 우선 구현
일정 과다 낙관으로 후반 작업 몰림	High	Extreme	Extreme	초기 범위 축소, 중간점검 강화, 날짜상의 버퍼 확보
문서화 부족으로 후속작업 차질	Medium	High	Extreme	Notion/Figma 활용 문서화, 산출물 관리 철저
사용자 요구와 결과 불일치	Medium	High	Extreme	프로토타입 공유, 중간 피드백 수렴
프론트-백엔드 API 불일치	Medium	High	Extreme	Swagger/REST Docs 문서화, 조기 통합 테스트
UX 부족으로 사용성 저하	Medium	Medium	High	Figma 시안 검토, 개발 도중 사용자 테스트
추가 기능 욕심으로 범위 확대	Low	Low	Low	MVP 범위 고수, 추가 기능은 보류, 리스트화
교수님의 기능 추가 요구	Medium	High	Extreme	변경 요청 문서화, 우선순위 재조정
OpenAI API 제한/지연	Low	Medium	Medium	대체 API 검토, 캐싱·로컬 fallback 기능
AWS 서비스 불안정	Low	High	Medium	로컬 환경 준비, 리허설 반복, 다중 리전 고려
테스트 부족으로 버그 다수 발생	Medium	High	Extreme	JUnit/Vitest 활용 자동화 테스트, CI/CD 연동
평가 결과 낮음	Medium	High	Extreme	사전 모의 평가, 품질 기준 수립
팀 내 갈등·의사소통 문제	Medium	Low	Medium	주 1회 회의, 협업 도구 적극 활용, 규칙 합의, 일정 공유

5. 개발 환경

가. SW 명세

◇ 프론트엔드

- 프레임워크 : Next.js(React 기반) / Tailwind
- 언어 : HTML / CSS / TypeScript(5.6.x)
- 테스트도구 : Vitest
- 코드품질 : Prettier/Eslint

◇백엔드

- 프레임워크 : SpringBoot(3.X)
- 언어 : Java17
- DBMS : MySQL
- 테스트 : JUnit5
- 빌드도구 : Gradle(9.0.0)
- 코드품질 : spotless / Checkstyle(google-java-format)
- IDE : IntelliJ
- API 문서화 : Swagger / Spring REST Docs

◇AI

- Grok / 오픈소스 LLM

◇협업도구

- git / github / Discord / Notion / figma

나. HW 명세

◇개발 환경(개발자 PC/노트북)

- CPU: Intel i5 이상 또는 Apple M1 이상
- RAM: 16GB 이상
- Storage: SSD 512GB 이상
- OS: Windows 11, macOS, Ubuntu 22.04 LTS
- 입력 장치 : 노트북 내장 마이크, 외장 USB 마이크(PC 사용 시 필요)

◇운영 환경(클라우드)

- 컨테이너화 : Docker (개발/운영 환경 일관성)
- 프론트엔드 : Vercel (Next.js 공식 지원, 자동 배포/프리뷰 제공)
- 백엔드 : AWS EC2 t3g.small (2vCPU, 2GB RAM, SSD 20–30GB)
- 데이터베이스 : AWS RDS for MySQL db.t3g.micro (2vCPU, 1GB RAM, SSD 20GB)

◇네트워크/보안

- AWS VPC 및 보안그룹 설정 (HTTP/HTTPS/SSH 포트만 허용)
- 도메인: Route53 또는 외부 등록 서비스

◇CI/CD

- GitHub Actions (자동 빌드/테스트/배포 파이프라인)

6. 개발사업비 산정내역서

(단위 : 천원)

항 목 (품명, 규격)		수 량	단 가	금 액			비 고
				계	현금		
인건비	응용SW 개발자	3 x 3 x 80 (시간)	42	30,240	30,240		3인 × 3개월 x 월 평균 근무 시간 × 시간 평균 임금
하드웨어	MacBook Air	3	1,500	4,500	4,500		
소프트웨어/도구 사용료	클라우드 서버 (AWS EC2 + RDS)	3	120	360	360		t3.medium + db.t3.micro (온디맨드, 서울 리전)
	Vercel Pro Plan	3 x 3	30	270	270		3인 × 3개월
	Grok API	3	5	15	15		월 30만 토큰 가정
	도메인/호스팅	1	12	12	12		1년 기준
	Notion plus	3	17	51	51		
	Github team	3	6	18	18		
기타 경비	회의비/자료 인쇄비	-	-	300	300		회의, 보고서, 발표자료
	예비비 (총액의 5%)			1,500	1,500		
총 합계				37,266	37,266		

7. 보고 및 모니터링 체계

가. 온라인/오프라인 회의 일정 계획

원활한 프로젝트 진행과 팀원 간의 긴밀한 소통을 위해 다음과 같이 회의 운영 계획을 수립합니다.

회의	주요안건	일시	장소	활용도구
주간정기회의	-지난 주 진행상황 리뷰 -금주 계획 수립 및 업무 분배 -이슈 및 리스크 논의 -보고서 작성 -발표 준비	매주 월요일 19:00~21:00	온라인 또는 오프라인(정보기술관 라운지)	Discord,Notion
수시/회의	-간단한 의사결정 -빠른 이슈 해결	필요시 수시로 진행	온라인	Discord

- Discord: 실시간 음성/화면 공유를 통한 회의 진행
- Notion: 회의록 작성 및 관리, 안건 사전 공유

나. Task 관리 시스템(Notion 데이터베이스)

프로젝트의 모든 작업은 Notion의 [전체 Task 관리] 데이터베이스로 관리

◇데이터베이스 속성 정의

속성 (Property)	타입 (Type)	설명	예시
Task ID	텍스트	고유 식별 번호. Github 브랜치명과 연동.	FE-001, BE-007
작업명	제목	수행할 작업을 명확하고 간결하게 기술	로그인 페이지 UI 구현
담당자	사람	해당 Task를 책임지고 수행할 팀원	@홍길동
분야	선택	작업의 기술 영역을 구분	Frontend, Backend, AI, Infra
상태	상태	Task의 현재 진행 상황 (칸반 보드와 연동)	대기, 진행 중, 리뷰 요청, 완료
마감일	날짜	Task 완료를 목표로 하는 날짜 (Deadlines)	2025-10-20
우선순위	선택	작업의 중요도 및 긴급도	P1(긴급), P2(높음), P3(보통)

◇Task 넘버링 규칙

- 각 Task는 분야별 Prefix와 순차 번호를 조합하여 고유 ID를 부여
  - 프론트엔드 : FE-001, FE-002, ...
  - 백엔드 : BE-001, BE-002, ...
  - AI : AI-001, AI-002, ...
  - 인프라 : IN-001, IN-002, ...

다. 산출물 관리

◇commit convention

- Commit Convention이란 팀원들이 Git 커밋 메시지를 일관된 형식으로 작성하기 위한 규칙을 의미
- 이를 통해 변경 이력을 명확하게 파악하고, 가독성을 높여 협업과 유지보수를 효율적으로 할 수 있음
- commit convention은 다음과 같은 형식을 따름.

타입	의미	설명	예시
add	파일 추가	기존에 없던 기능이나 파일을 새로 추가할 때 사용	add: 이미지 업로드 기능 추가 (#TASK-102)
delete	기능 삭제	불필요한 파일이나 코드, 기능을 삭제할 때 사용	delete: 사용하지 않는 API 엔드포인트 삭제 (#TASK-103)
feat	새로운 기능 추가	새로운 기능이나 모듈을 개발할 때 사용	feat: 회원가입 기능 추가 (#TASK-100)
fix	버그 수정	발견된 오류나 버그를 해결할 때 사용	fix: 로그인 오류 수정(#TASK-101)
docs	문서 변경	코드가 아닌 문서(README, 위키 등)를 수정할 때 사용	docs: API 문서 업데이트

style	코드 스타일 변경	코드 로직에 영향 없는 세미콜론, 공백, 네이밍 수정	style: 불필요한 공백 제거
refactor	코드 리팩토링	기능 변화 없이 구조나 가독성을 개선할 때 사용	refactor: 유저 인증 로직 분리
test	테스트 코드	테스트 케이스 추가, 수정, 보강 시 사용	test: 회원가입 단위 테스트 추가
chore	빌드/설정 변경	패키지 매니저, 빌드 설정, 의존성 업데이트 등	chore: Gradle 버전 업데이트
perf	성능 개선	성능 최적화 목적의 변경 사항	perf: DB 쿼리 속도 개선
hotfix	긴급 수정	배포 후 긴급하게 수정이 필요한 버그나 문제 해결	hotfix: 배포 후 결제 시스템 오류 수정 (#TASK-104)

◇브랜치 명명 규칙

- 브랜치 명명 규칙은 협업 효율성과 코드 관리를 위해 Git 브랜치의 이름을 정하는 일관된 규칙
- 소문자, 하이픈, 슬래시를 사용하여 기능별, 이슈별로 브랜치를 체계화

브랜치 타입	의미	설명	예시
feature/	새로운 기능 개발	특정 기능 구현 단위로 브랜치를 생성	feature/TASK-101-회원가입
bugfix/	일반 버그 수정	QA나 운영 중 발견된 버그를 수정할 때 사용	bugfix/TASK-202-로그인실패
hotfix/	긴급 버그 수정	서비스 장애, 치명적 버그 즉시 대응	hotfix/TASK-303-결제에러
release/	배포 준비용	정식 배포 전 QA, 안정화 작업	release/TASK-400-v1.2.0
experiment/	실험적 코드	새로운 시도, PoC(Proof of Concept) 작업용	experiment/TASK-500-신규UI

◇Figma: UI/UX 디자인 산출물

모든 시각적 결과물을 Figma를 통해 만들고 공유

- 관리 대상
  - 와이어프레임 (Wireframe): 화면의 초기 구조 설계
  - UI 디자인 (Mockup): 실제 구현될 화면의 상세 디자인
  - 프로토타입 (Prototype): 사용자 흐름을 확인할 수 있는 인터랙티브 목업
  - 디자인 시스템: 버튼, 컬러, 폰트 등 재사용 가능한 UI 컴포넌트 정의

라. 진행 상황 모니터링 및 보고 (Monitoring & Reporting)

◇모니터링 방식

- PM이 주관하는 매주 월요일 정기 회의 시간에 Notion 보드를 기반으로 공식적인 진행 상황을 리뷰
- 팀원들은 주간 진행 보고 전까지 자신의 Task 상태 업데이트를 완료

◇주간 진행 보고 (Weekly Report)

- 정기 회의에서 PM은 Notion 화면을 공유하며 회의를 주도하고, Task 리뷰, 마감일 점검, 팀원별 부하 상태, 차주 계획 수립의 내용을 팀원들과 함께 보고 및 논의